# Extended Abstract

**Motivation** Robots operating in human environments need reusable manipulation primitives, such as pushing, reaching, and pick-and-place, that a higher-level planner can invoke and *sequence* rather than relearn for every task. Learning these primitives without demonstrations is difficult because manipulation is contact-rich, sparse-reward, and sensitive to unstable actions; *composing* them into longer behaviors adds a second difficulty, because online exploration over several primitives can wander outside the region where any of them is reliable. We study whether contrastive reinforcement learning (CRL) can provide a demo-free foundation for reusable manipulation skills, and whether a single mechanism—constraining a policy to a Mahalanobis ball around a primitive's learned action distribution—can both *refine* a frozen primitive and *compose* several into a new task.

**Method** Our method centers on one constraint applied two ways. We first train goal-conditioned manipulation primitives with CRL, whose critic scores compatibility between a state-action pair and a goal through learned embeddings trained with an InfoNCE-style objective and hindsight relabeling—learning from sparse goal-reaching feedback without demonstrations or dense reward engineering. *Refine:* we freeze a learned primitive and train a small residual correction, constrained by a Mahalanobis Action Barrier (MAB) in either raw action space or the critic's learned embedding space, so the residual cannot override the primitive. *Compose:* on a mobile manipulator, we keep several frozen primitives, learn a gate that selects among them, and add a bounded residual whose exploration is confined to the selected primitive's Mahalanobis ball, optimizing a new task's sparse reward by policy gradient. The same ball—its metric set by a primitive's own action covariance—bounds a correction in the first case and bounds exploration in the second.

**Implementation** We evaluate in MuJoCo. On a fixed-base Franka Panda arm, `arm_push_easy` validates CRL against standard RL baselines and `arm_push_hard` evaluates MAB residual refinement on a frozen CRL primitive. We then build a TidyBot mobile-manipulation suite in MuJoCo MJX—an 11-DoF holonomic base, Kinova arm, and gripper—train a demo-free CRL *push-aside* primitive, and target a novel *hallway* task: drive down a corridor, push a blocking cube aside, and pass, with a success criterion that rejects bulldozing. Bringing CRL primitives onto the mobile base surfaced two concrete simulation bugs that we diagnosed and fixed: an infinite-range action-conversion NaN from the arm's continuous-rotation joints, and a CPU-versus-GPU MJX instability caused by an unphysically small base inertia. A scripted sequencer over the demo-free CRL push-aside primitive solves the hallway in most seeds; a learned constrained-exploration composition over the primitives is in progress.

**Results** On `arm_push_easy`, CRL is the strongest base learner ($0.90$ success, $120.9$ mean steps) versus SAC ($0.50$, $509.7$) and vanilla policy gradient ($0.00$). On `arm_push_hard`, embedding-space MAB improves mean success from $0.672$ for the frozen CRL policy to $0.714$ at the final checkpoint ($0.755$ at the best), preserving the approach-and-push structure while making local corrections. On TidyBot, a scripted navigate→push-aside→pass sequencer built on the demo-free CRL push primitive clears the obstacle and reaches the goal in five of six seeds; the learned-gate composition runs end-to-end but has not yet converged. Diagnosing the mobile setting also produced two reproducible simulation findings (the infinite-range NaN and a base-inertia GPU overflow).

**Discussion** CRL is a strong demo-free base, while MAB is best understood as a constraint that keeps a policy near a primitive's reliable action geometry. As *refinement*, it improves success near the base primitive's training distribution but does not consistently improve efficiency or smoothness, is sensitive to checkpoint selection, and is bounded by how well the frozen critic, actor, and embedding generalize out of distribution. As *composition*, constraining exploration to the primitives' balls lets the policy draw only on motions the primitives already know, but the bottleneck shifts to high-level credit assignment: learning *when* to switch primitives from sparse reward is hard, and our preliminary gate learns slowly. The two simulation findings underscore how brittle contact-rich whole-body simulation can be, and in particular how the same model can be stable on CPU yet diverge on GPU.

**Conclusion** This project shows that contrastive goal-conditioned learning produces useful demo-free pushing primitives on both a fixed arm and a mobile manipulator, and that a Mahalanobis ball

around a primitive's learned action distribution is a single mechanism for both locally refining a frozen primitive and constraining exploration when composing several. Scaling either direction will likely require broader training distributions, stronger representations, better uncertainty estimates, and—for composition—stronger high-level credit assignment than sparse-reward policy gradient provides.

# Refine and Compose: Mahalanobis Action Barriers over Demo-free Contrastive RL Primitives

**Dylan Zhou**
Department of Computer Science
Stanford University
dylan23@stanford.edu

**Anuva Banwasi**
Department of Computer Science
Stanford University
anuva@stanford.edu

**Jiaye Zou**
Department of Computer Science
Stanford University
tonyzou@stanford.edu

## Abstract

We study demo-free learning *and composition* of reusable robot manipulation primitives using contrastive reinforcement learning (CRL). CRL trains a goal-conditioned critic with a contrastive objective and hindsight relabeling, enabling sparse-reward manipulation without demonstrations or dense reward engineering. Throughout, we use a single mechanism in two regimes: a *Mahalanobis Action Barrier* (MAB) that constrains a policy to a ball around a CRL primitive's learned action distribution. We first validate CRL on a MuJoCo Franka arm, where it outperforms SAC and vanilla policy gradient on `arm_push_easy`. We then use MAB to *refine* a single frozen primitive: on `arm_push_hard`, embedding-space MAB raises mean success from $0.672$ to $0.714$ at the final checkpoint, but does not consistently improve efficiency, smoothness, or transfer—we find its gains are bounded by the generalization of the learned critic, actor, and embedding geometry. We then extend CRL primitives to a *TidyBot mobile manipulator* and use MAB to *compose* rather than refine them, on a novel hallway task: navigate to a blocking object, push it aside, and pass. A scripted sequencer over a demo-free CRL push-aside primitive clears the obstacle and reaches the goal in five of six seeds, and we present a constrained-exploration composition method—a learned gate plus a bounded residual confined to the primitives' Mahalanobis balls—together with preliminary learning results. We additionally report two concrete simulation findings, including a CPU-versus-GPU MuJoCo MJX instability traced to an unphysical base inertia. Together, these results position CRL as a strong base for demo-free primitives, and constrained exploration over their learned action geometry as a promising—though generalization-limited—tool for both refining and composing them.

## 1 Introduction

Robots that operate in human environments need reusable manipulation skills. A household robot should not have to relearn pushing, reaching, opening, or pick-and-place behavior from scratch for every new task. Instead, it should acquire general-purpose primitives that can be invoked and composed by a higher-level planner. Learning such primitives, however, is difficult because robot manipulation is long-horizon, contact-rich, sparse-reward, and sensitive to action instability. Demonstrations can reduce this difficulty, but they are expensive to collect, task-specific, and hard to scale across embodiments and environments.

Stanford CS224R 2026 Final Report

In this project, we study demo-free robot primitive learning through *contrastive reinforcement learning* (CRL). CRL frames goal-conditioned reinforcement learning as a contrastive prediction problem: the agent learns representations that bring reachable state-action pairs closer to goal states while pushing apart unrelated goals (Eysenbach et al., 2022). Recent work has shown that scaling this approach with deeper networks can improve self-supervised goal-reaching behavior in simulated locomotion and manipulation domains (Wang et al., 2025). This makes CRL a promising foundation for learning reusable manipulation primitives from sparse reward and hindsight relabeling, without human demonstrations or dense reward engineering.

We first validate whether CRL can learn reliable manipulation primitives in our setting. We reproduce and extend CRL-style training in MuJoCo, focusing on a Franka Panda arm pushing a cube from a start region to a goal region. Compared with standard reinforcement learning baselines such as SAC and vanilla policy gradient, CRL learns more successful closed-loop behavior under sparse rewards. We then study whether a frozen CRL primitive can be improved through constrained residual correction. A pretrained primitive may work well on its training distribution but fail under changes in object pose, goal location, contact dynamics, or robot embodiment. To address this, we investigate a *Mahalanobis Action Barrier* (MAB), which constrains a learned residual correction to remain close to the base primitive either in raw action space or in the CRL critic's learned embedding space.

Finally, we ask whether the *same* Mahalanobis-ball constraint supports *composing* primitives, not just refining one. We move to a simulated TidyBot mobile manipulator and a "hallway" task that requires sequencing two primitives—navigate and push-aside—to drive past a blocking object. We learn a gate that selects among frozen primitives together with a bounded residual whose exploration is confined to balls around each primitive's learned action distribution, so the composite policy can only combine motions the primitives already know. In both regimes the same construction appears: a Mahalanobis ball whose metric is set by a primitive's own action covariance, used to bound a correction when refining and to bound exploration when composing.

Our experiments show a mixed but informative result. CRL is effective for learning the base pushing primitive and outperforms the standard RL baselines we tested. As *refinement*, MAB modestly improves the arm-push task but does not consistently transfer to harder settings such as pick-and-place or out-of-distribution start and goal configurations, because it depends on the frozen critic, actor, and embedding geometry generalizing. As *composition*, a scripted sequencer over a demo-free CRL push-aside primitive solves the TidyBot hallway in most seeds, and a constrained-exploration composition trains end-to-end, but learning *when* to switch primitives from sparse reward is hard and our results there remain preliminary. Our contributions are: (i) an empirical validation of CRL for demo-free primitive learning against SAC and vanilla policy gradient; (ii) the Mahalanobis Action Barrier as a single mechanism for both refining a frozen primitive and composing several; (iii) a TidyBot mobile-manipulation suite, a demo-free push-aside primitive, and a working scripted hallway demonstration, with a constrained-exploration composition method; and (iv) two reproducible MuJoCo MJX simulation findings encountered in bringing CRL primitives onto a mobile base.

## 2 Related Work

**Goal-conditioned and contrastive reinforcement learning.** Goal-conditioned reinforcement learning aims to learn policies that can reach many possible goals rather than solve a single fixed task (Schaul et al., 2015; Andrychowicz et al., 2017). This formulation is attractive for robotics because a single learned policy can potentially support many downstream instructions by changing the goal input. However, sparse goal-reaching tasks are difficult because the agent receives useful feedback only when it reaches the commanded goal. Hindsight experience replay partially addresses this issue by relabeling failed trajectories with goals that were actually achieved (Andrychowicz et al., 2017). Contrastive RL builds on this idea by training a critic to distinguish compatible state-action–goal tuples from incompatible ones using a contrastive objective such as InfoNCE (Eysenbach et al., 2022). Recent scaling work shows that deeper networks can improve self-supervised goal-reaching performance in simulated control domains (Wang et al., 2025). We use CRL as our base demo-free primitive-learning method.

**Standard RL baselines for continuous control.** Continuous-control robot learning is often evaluated against actor-critic methods such as soft actor-critic (SAC) (Haarnoja et al., 2018) and policy-gradient methods such as vanilla policy gradient or PPO-style algorithms (Williams, 1992; Schulman

et al., 2017). These methods are general and widely used, but they can struggle in sparse-reward manipulation without reward shaping, demonstrations, or curriculum design. In our experiments, CRL outperforms SAC and vanilla policy gradient on the arm-push task, motivating our use of contrastive goal-conditioned learning as the base primitive.

**Residual policy learning and constrained correction.** Residual policy learning improves an existing controller or policy by learning an additive correction (Silver et al., 2018; Johannink et al., 2019). This is appealing for robotics because the base controller provides reasonable behavior while the residual adapts it to new dynamics, contacts, or task variations. However, unconstrained residuals can also destabilize behavior by moving the policy far from the distribution where the base controller is reliable. Our MAB method follows the residual-learning idea but constrains the residual using a state-dependent Mahalanobis metric. It is also related to trust-region methods, which restrict policy changes during optimization (Schulman et al., 2015, 2017). Unlike standard trust-region methods, MAB constrains the residual action correction itself, either in action space or in the CRL critic's learned embedding space.

**Skill composition, options, and behavior-constrained RL.** Composing reusable skills into longer behaviors is the subject of hierarchical RL and the options framework, where a policy over options selects temporally extended sub-policies (Sutton et al., 1999; Bacon et al., 2017). Our composition setting is an instance of this: a learned gate selects among frozen primitives and commits to each for several steps. A second ingredient is keeping the composite policy within behaviors the primitives actually support. Behavior-constrained methods from batch and offline RL, such as BCQ and BEAR, restrict the policy to actions near a data distribution to avoid extrapolation error (Fujimoto et al., 2019; Kumar et al., 2019). Latent- and action-barrier methods make this constraint geometric: recent humanoid-control work bounds actions within a Mahalanobis ball in a latent space learned from human motion data, so the agent only deviates within demonstrated behavior (Zhang et al., 2026). We adopt the same Mahalanobis-ball construction, but place it around each CRL primitive's own *demo-free* action distribution and reuse it for both refining a single primitive and constraining exploration when composing several.

**Robot primitives and whole-body manipulation.** Reusable primitives such as pushing, grasping, opening, and placing are useful building blocks for long-horizon robot behavior. Whole-body mobile manipulation adds further difficulty because the robot must coordinate base motion, arm motion, and contact dynamics. In this project, we study both a fixed-base Franka pushing primitive and a simulated TidyBot mobile manipulator, on which we train a demo-free push-aside primitive and compose primitives to solve a hallway task. This setting exposes the instability and coupling introduced by whole-body control, which we encounter as both a learning challenge (high-level credit assignment for composition) and a simulation challenge (contact-rich dynamics that can diverge across compute backends).

## 3 Method

Our method has two stages. First, we train a goal-conditioned manipulation primitive using contrastive reinforcement learning (CRL). Second, we freeze this primitive and train a small residual policy to correct its actions. To prevent the residual from overriding the learned behavior, we constrain it with a Mahalanobis Action Barrier (MAB). We consider two variants: action-space MAB, which bounds the residual in raw action coordinates, and embedding-space MAB, which bounds the correction in the CRL critic's learned state-action representation.

### 3.1 Problem Setup

We consider goal-conditioned manipulation tasks where the robot observes a state $s_t \in \mathcal{S}$, receives a goal $g \in \mathcal{G}$, and outputs a continuous action $a_t \in \mathcal{A}$. The objective is to reach the goal using sparse task feedback. In our main environment, a simulated Franka Panda arm must push a cube from a start region to a goal region. The policy must learn closed-loop behavior from interaction alone, without demonstrations or dense reward shaping.

### 3.2 Contrastive Reinforcement Learning Primitive

CRL learns a critic that scores whether a state-action pair is compatible with a goal. Following prior work (Eysenbach et al., 2022; Wang et al., 2025), we parameterize the critic using a state-action embedding $\phi_\theta(s, a)$ and a goal embedding $\psi_\theta(g)$:

$$f_\theta(s, a, g) = -\left\|\phi_\theta(s, a) - \psi_\theta(g)\right\|_2 . \tag{1}$$

The critic assigns higher scores when the state-action embedding is close to the goal embedding. Given a batch of transitions and relabeled goals, we train it with an InfoNCE-style objective:

$$\mathcal{L}_{\text{CRL}} = -\log \frac{\exp(f_\theta(s_i, a_i, g_i))}{\sum_{j=1}^{B} \exp(f_\theta(s_i, a_i, g_j))}, \tag{2}$$

where $g_i$ is a positive goal associated with transition $(s_i, a_i)$ and the remaining goals in the batch act as negatives. We use hindsight relabeling by treating states reached later in the same trajectory as achieved goals.

The policy is trained to choose actions that maximize the learned contrastive critic:

$$\max_\pi \mathbb{E}_{s,g,a\sim\pi}\left[f_\theta(s, a, g)\right]. \tag{3}$$

For the arm-push task, this produces a closed-loop primitive that repeatedly observes the cube and goal locations and adjusts its action to push the cube toward the target.

### 3.3 Residual Correction

A frozen CRL primitive may be competent on its training distribution but brittle under changes in object pose, goal location, contact dynamics, or embodiment. To adapt the primitive without retraining it, we learn a residual correction on top of the frozen base policy. Let $\pi_{\text{base}}$ denote the pretrained CRL policy:

$$a_{\text{base}} = \pi_{\text{base}}(s, g). \tag{4}$$

A residual policy $r_\eta(s, g)$ proposes a correction, giving

$$a = a_{\text{base}} + r_\eta(s, g). \tag{5}$$

We parameterize $r_\eta$ as a small MLP over the observation $(s, g)$, mirroring the base actor's architecture but with a zero-initialized output layer. Consequently $r_\eta \equiv 0$ at initialization, so the corrected policy begins exactly at $\pi_{\text{base}}$ and any subsequent departure is attributable solely to the learned residual.

The base policy and critic remain frozen during residual training, so any improvement comes from the residual rather than from further training the original CRL primitive. Because unconstrained residuals can move the action outside the distribution where the base primitive is reliable, we constrain the correction with MAB.

### 3.4 Mahalanobis Action Barrier

MAB constrains residual corrections using a state-dependent covariance. The core idea is that residuals should be small in directions where the base primitive is confident and more flexible in directions where the base primitive naturally varies. A Mahalanobis constraint captures this geometry better than a uniform Euclidean bound. This choice is especially well-motivated for CRL: the critic's embeddings are trained with an InfoNCE objective, and recent analysis shows that InfoNCE drives representations toward an approximately Gaussian distribution on the hypersphere (Betser et al., 2026). Because the Mahalanobis distance is the natural metric of a Gaussian, constraining corrections in the critic's learned embedding geometry matches the distribution the contrastive objective actually induces—a justification specific to contrastive critics, in contrast to the empirically-motivated barriers used in prior latent-action work.

**Action-space MAB.** In action-space MAB, the residual is constrained directly in raw action space using a covariance $\Sigma_a(s)$ around the base action:

$$r_\eta(s, g)^\top \Sigma_a(s)^{-1} r_\eta(s, g) \le \epsilon_a^2. \tag{6}$$

If the proposed residual violates the constraint, we scale it back to the boundary of the Mahalanobis ball:

$$\tilde{r}_\eta(s,g) = \frac{\epsilon_a}{\max\left(\epsilon_a, \sqrt{r_\eta(s,g)^\top \Sigma_a(s)^{-1} r_\eta(s,g)}\right)} r_\eta(s,g), \tag{7}$$

and execute $a = a_{\text{base}} + \tilde{r}_\eta(s,g)$.

**Embedding-space MAB.** Embedding-space MAB instead constrains the correction in the CRL critic's learned representation. Since the critic evaluates actions through $\phi_\theta(s,a)$, we measure how much the corrected action changes the state-action embedding:

$$\Delta\phi(s) = \phi_\theta(s,a) - \phi_\theta(s, a_{\text{base}}). \tag{8}$$

The embedding-space constraint is

$$\Delta\phi(s)^\top \Sigma_\phi(s)^{-1} \Delta\phi(s) \le \epsilon_\phi^2. \tag{9}$$

This keeps the corrected action close to the base action according to the critic's own learned geometry, rather than only in motor-command space. This distinction is useful because small raw action changes can produce large representation changes near contact, while larger raw action changes may still preserve the same goal-directed behavior.

### 3.5 From Refining One Primitive to Composing Many

The same Mahalanobis ball that *refines* a single frozen primitive can constrain *exploration* when *composing* several. We instantiate this on a mobile manipulator that must solve a new task by combining frozen primitives rather than learning from scratch. Our target is a hallway task: the robot drives down a corridor toward a goal, but a cube blocks the lane; success requires the base to reach the goal *and* the cube to be cleared aside (or lifted), a criterion that rejects simply bulldozing the cube forward. The composite policy combines a *navigate* primitive and the CRL *push-aside* primitive. Only a small high-level policy is learned; the primitives and their critics stay frozen.

**Primitives as providers.** At each step, every primitive $k$ exposes a center and a per-dimension scale in the shared 11-DoF action space, $(\mu_k(s), \sigma_k(s))$. For a CRL primitive we query its frozen Gaussian actor at that primitive's own synthesized goal and read $\mu_k = \tanh(\text{mean})$ and $\sigma_k = \exp(\text{log\_std})$—i.e. its learned action distribution. For the *navigate* primitive (navigation is treated as a classical, non-learned planner skill) the center is a base controller and $\sigma_k$ is a fixed diagonal scale on the base dimensions. The per-primitive goals are produced by a hand-specified planner interface, standing in for the role a vision-language-model planner would play in a full system.

**Gated, ball-constrained action.** A learned gate $\rho_\zeta(s)$ outputs selection logits and a raw residual $\mathbf{r}$. It selects a primitive $k^\star$ and projects the residual into that primitive's Mahalanobis ball:

$$\mathbf{u} = \epsilon \frac{\mathbf{r}}{\max(1, \|\mathbf{r}\|)}, \qquad a = \text{clip}\big(\mu_{k^\star}(s) + \sigma_{k^\star}(s) \odot \mathbf{u},\, -1,\, 1\big). \tag{10}$$

The correction is bounded by $\epsilon$ in the diagonal Mahalanobis metric set by $\sigma_{k^\star}$, so the composite policy stays within motions the selected primitive supports. We compare a barrier ladder that isolates each ingredient: `none` (unconstrained RL, $a = \tanh(\mathbf{r})$), `single` (one fixed primitive's ball, no selection), `euclidean` (gated but $\sigma \equiv \mathbf{1}$), and `multi` (the gated per-primitive diagonal-Mahalanobis ball).

**Temporal abstraction and learning.** The task reward is sparse hallway success; we optimize the gate and residual with REINFORCE. Two choices proved essential. First, *gate stickiness*: the gate commits to a selected primitive for $K$ steps before re-selecting, so a zero-initialized gate produces coherent navigate→push→navigate executions instead of jittering between primitives every step. The gate's policy gradient is counted only at re-selection steps and normalized by their count, so the sparse selection signal is not diluted by the committed steps. Second, *reward-to-go* credit assignment: rather than crediting every timestep with the whole-episode return (which reinforces every choice in every state equally), each step is credited with its discounted future return $R_t = \sum_{t' \ge t} \gamma^{t'-t} r_{t'}$, so the decisions that lead into success receive the credit.

### 3.6 Training Procedure and Implementation

Training proceeds in two phases. We first train the CRL primitive using sparse goal-reaching reward and hindsight relabeling. We then freeze the learned policy and critic, initialize a residual policy, and train only the residual under the MAB constraint. We compare the frozen CRL primitive against residual variants with action-space and embedding-space MAB.

Our Franka refinement experiments use a MuJoCo Panda arm on `arm_push_easy` and `arm_push_hard`, where the robot pushes a cube from a start region to a goal region. For the composition study we build a TidyBot suite in MuJoCo MJX with an 11-DoF holonomic base, Kinova arm, and parallel-jaw gripper: a `tidybot_push_aside` primitive (a frozen-base lateral push, trained demo-free with CRL), a classical `tidybot_navigate` base controller, and the composite `tidybot_hallway` task. To deploy the base-frozen push primitive inside the mobile hallway, we apply a base-relative observation transform so the primitive sees the cube where it was trained. We then optimize the gate and residual with the reward-to-go REINFORCE objective above, holding the primitives frozen throughout.

## 4 Experimental Setup

We evaluate three questions. First, can contrastive reinforcement learning learn a useful demo-free manipulation primitive under sparse rewards? Second, can a Mahalanobis-constrained residual *refine* a frozen CRL primitive without destabilizing it? Third, can the same Mahalanobis-ball construction support *composing* primitives into a new task on a mobile manipulator? We study the first two on a simulated Franka Panda arm and the third on a simulated TidyBot mobile manipulator, both in MuJoCo.

### 4.1 Environments and Tasks

Our main environment is a fixed-base Franka Panda manipulation task. The robot must push a cube from an initial region to a goal region using continuous arm actions. We use `arm_push_easy` to validate that CRL can learn a sparse-reward manipulation primitive, and `arm_push_hard` to evaluate whether a frozen CRL primitive can be improved using MAB residual correction.

We also tested harder settings, including pick-and-place and out-of-distribution start or goal configurations. These tasks require more precise contact behavior and stronger generalization from the learned critic and actor.

For the composition study we use a TidyBot mobile manipulator in MuJoCo MJX with an 11-DoF action space (holonomic base, 7-DoF Kinova arm, parallel-jaw gripper). We define three environments. `tidybot_push_aside` is a frozen-base lateral push: the arm starts near contact height and must nudge a cube to a short lateral offset, with sparse $xy$-distance success; we train it demo-free with CRL. `tidybot_navigate` is a base-only drive-to-target task that defines the classical navigation controller. `tidybot_hallway` is the composite task: a corridor with side walls, a cube blocking the lane, and a far-end goal. Its success criterion requires the base to reach the goal *and* the cube to be pushed aside ($|x|$ beyond a threshold) or lifted—an anti-bulldoze condition that rejects the trivial solution of shoving the cube straight ahead. The cube's initial position is randomized so the composition must generalize across layouts.

### 4.2 Baselines and Variants

We compare CRL against standard continuous-control RL baselines on the easier pushing task. The baselines are vanilla policy gradient and soft actor-critic (SAC) (Haarnoja et al., 2018). For the harder pushing task, we compare the frozen CRL base policy against CRL with MAB residual correction. Our reported residual results use embedding-space MAB; action-space MAB is included as an alternative formulation but was not the strongest variant in our experiments. For the composition task, we compare the barrier ladder of Section 3.5—`none`, `single`, `euclidean`, and `multi`—to isolate the effects of gated selection and of the per-primitive Mahalanobis metric, and we use a scripted sequencer over the same frozen primitives as a non-learned reference for the intended composite behavior.

### 4.3 Metrics

We report success rate, steps or time to completion, and action smoothness. Success rate measures whether the robot completes the task. Steps and time to completion measure efficiency, where lower is better. For residual correction, we also evaluate action smoothness using first-order action velocity and second-order action difference statistics. These metrics help distinguish whether MAB improves task performance while preserving stable control. For the composition task, we report the hallway success rate, the clearance rate of the scripted sequencer across seeds, and the gate's primitive-selection distribution, which indicates whether the learned policy acquires a navigate→push→navigate sequence rather than collapsing onto a single primitive.

## 5 Results

### 5.1 CRL Learns the Base Manipulation Primitive

We first compare CRL against standard RL baselines on `arm_push_easy`. CRL achieves the highest success rate and reaches the goal in fewer steps than SAC and vanilla policy gradient. SAC solves some rollouts but is less reliable, while vanilla policy gradient fails under the sparse-reward setting.

Table 1: Comparison of CRL against standard RL baselines on `arm_push_easy`. CRL achieves higher success and lower mean steps to success.

| **Method** | **Success Rate** ↑ | **Mean Steps** ↓ |
|---|---|---|
| Vanilla Policy Gradient | 0.00 | 1000.0 |
| SAC | 0.50 | 509.7 |
| CRL | **0.90** | **120.9** |

These results support our first hypothesis: contrastive goal-conditioned learning provides an effective signal for sparse-reward primitive learning. Hindsight relabeling and contrastive goal comparisons provide useful learning signal even when the originally commanded goal is not reached, allowing CRL to learn a reusable closed-loop pushing primitive.

### 5.2 MAB Improves Success on `arm_push_hard`

We next evaluate whether MAB can improve a frozen CRL primitive on `arm_push_hard`. The base CRL policy achieves a mean success rate of $0.672$ across three seeds. Adding MAB improves performance: the final corrected policy at iteration 20 reaches a mean success rate of $0.714$, while the best checkpoint across training reaches $0.755$.

Table 2: Success rate on `arm_push_hard` before and after MAB correction. MAB improves mean success over the frozen CRL base policy, though the gain is modest.

| **Method** | **Seed 0** | **Seed 1** | **Seed 2** | **Mean** |
|---|---|---|---|---|
| CRL base | 0.664 | 0.688 | 0.664 | 0.672 |
| MAB corrected, final iter. 20 | 0.727 | 0.711 | 0.703 | 0.714 |
| MAB corrected, best over iters. | **0.750** | **0.758** | **0.758** | **0.755** |
| $\Delta$ final $-$ base | +0.063 | +0.023 | +0.039 | +0.042 |

The improvement suggests that constrained residual correction can adapt a pretrained primitive without fully retraining it. The residual helps the frozen policy recover from some local errors, while the Mahalanobis constraint prevents the correction from moving too far from the base behavior. However, the gain is modest, so MAB is better understood as a stabilizing correction mechanism than as a complete solution to primitive generalization. The gap between the final and best checkpoints also shows that residual training is not monotonic, so checkpoint selection matters.

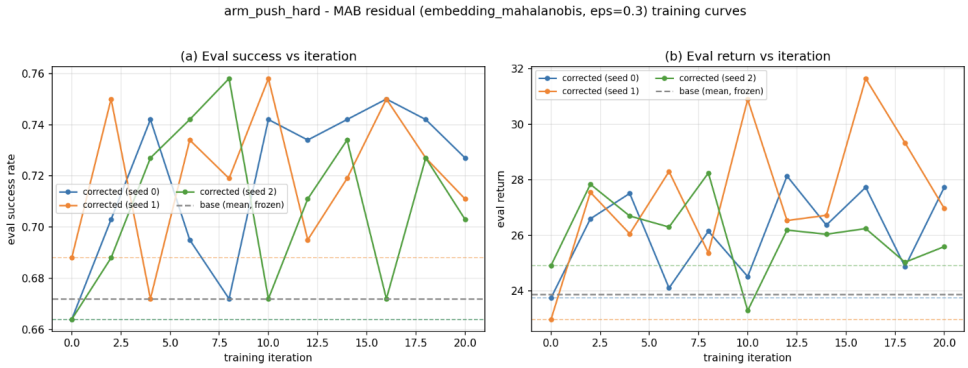


Figure 1: **MAB residual training on `arm_push_hard`.** We train an embedding-space MAB residual on a frozen CRL primitive for 20 iterations across three seeds. Solid curves show corrected policies; dashed lines show frozen base performance. MAB reaches a best mean success of $0.755$ versus $0.672$ for the base policy, but the non-monotonic curves suggest sensitivity to checkpoint selection.

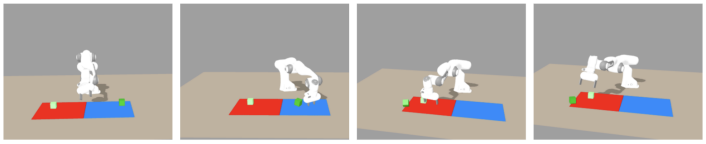

*Fig 2a. CRL pretrained rollout*

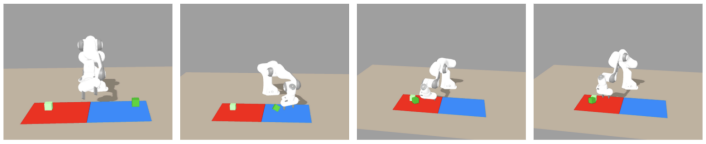

*Fig 2b. CRL w/ MAB correction rollout*

Figure 2: **Qualitative comparison of the frozen CRL primitive and MAB-corrected policy.** Top: rollout from the pretrained CRL policy. Bottom: rollout from the same primitive with embedding-space MAB residual correction. Both policies follow the same broad approach-and-push behavior, while MAB makes local adjustments to the interaction. This supports the intended role of MAB as a constrained correction mechanism rather than a replacement for the learned primitive.

### 5.3 MAB Preserves the Base Rollout Structure

Qualitatively, MAB-corrected rollouts preserve the broad structure of the pretrained CRL primitive. In both the base and corrected policies, the robot approaches the cube and pushes toward the goal region. Rather than producing a new behavior from scratch, MAB introduces local changes to the interaction, consistent with its role as a constrained residual correction around the frozen primitive.

### 5.4 Efficiency and Smoothness

We also compare the base CRL policy and MAB-corrected policy using time to completion, mean steps, and action smoothness. Although the corrected policy improves success on `arm_push_hard`, its effects on efficiency and smoothness are mixed: in some seeds MAB reduces time or steps, while in others it takes longer or produces higher action variation. First-order action velocity and second-order action difference show the same pattern, suggesting that the Mahalanobis constraint limits residual deviations but does not automatically guarantee smoother control.

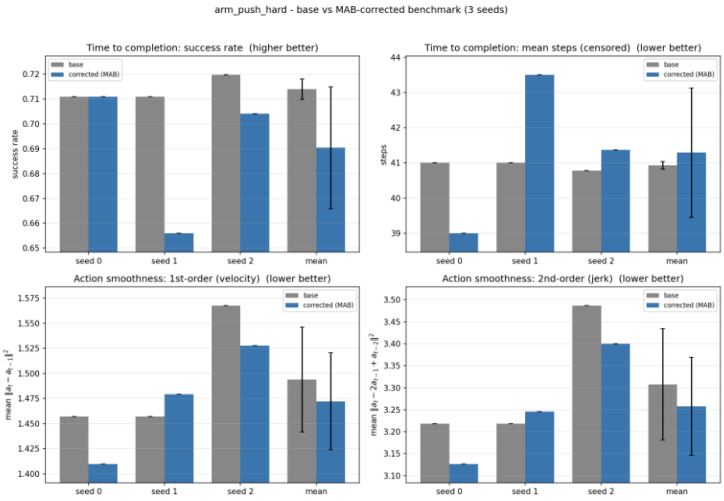


Figure 3: **Efficiency and smoothness comparison for base CRL vs. MAB correction.** Across three seeds on `arm_push_hard`, MAB improves success in some settings but has mixed effects on mean steps, first-order action velocity, and second-order action difference. The correction can improve task completion without consistently improving execution efficiency or smoothness, suggesting that the Mahalanobis constraint limits residual deviations but does not by itself guarantee smoother control.

### 5.5 Limitations on Harder Tasks

Although MAB improves `arm_push_hard`, we did not observe consistent success on harder tasks such as pick-and-place or out-of-distribution start and goal configurations. In these settings, the residual correction is limited by the quality of the frozen CRL critic and actor. If the base critic does not assign meaningful scores outside the training distribution, then the residual has little reliable signal for improvement. Similarly, if the base actor proposes actions that are already far from useful behavior, a constrained residual may not be able to recover.

This limitation is especially important for embedding-space MAB. Embedding-space MAB is useful only when the CRL embedding captures behaviorally meaningful distances. When the learned representation generalizes poorly, staying close in embedding space does not necessarily preserve useful behavior. MAB depends on the learned geometry of the base CRL primitive: it can stabilize corrections near the training distribution, but it cannot by itself fix a critic or representation that fails to generalize.

### 5.6 A Demo-free Push-Aside Primitive Composes into a Hallway Demonstration

We next move from *refining* a single primitive to *composing* several on the TidyBot mobile manipulator. We first train a `push_aside` primitive demo-free with CRL on the mobile base (frozen base, gripper closed), using the same sparse goal-reaching reward and hindsight relabeling as the Franka primitive. To use this base-frozen primitive inside the mobile hallway, we feed it a base-relative observation—subtracting the base pose so the cube appears where the primitive was trained—and hold the base while it acts.

We then verify that the frozen primitives *compose* mechanically using a scripted sequencer that drives to a standoff behind the cube, invokes the CRL push until the lane clears, and then drives to the far-end goal. Figure 4 shows a representative rollout. Across six seeds, the sequencer clears the cube and reaches the goal in **five of six seeds on GPU** (four of six on CPU); the failures are seeds in which the base does not settle into a clean push pose. This confirms that the demo-free push primitive transfers into the mobile task and that these two primitives suffice to solve the hallway—establishing the target behavior the learned composition should reproduce.

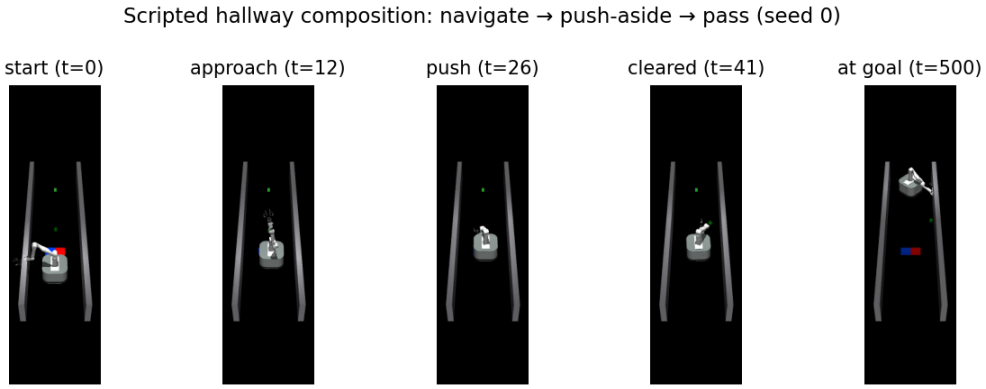


Figure 4: **Scripted hallway composition: navigate → push-aside → pass.** A scripted sequencer over the frozen primitives. The base drives down the corridor (*approach*), the demo-free CRL push-aside primitive nudges the blocking cube to the side (*push*, *cleared*), and the base then drives to the far-end goal (*at goal*). The cube is cleared from the lane ($|x|$ beyond the success threshold) rather than bulldozed forward. The frozen primitives compose to solve the hallway in five of six seeds on GPU.

### 5.7 Two Simulation Findings in the Mobile Setting

Bringing CRL primitives onto the mobile base surfaced two reproducible MuJoCo issues worth recording, each of which silently produced flat-zero training before it was diagnosed.

**Infinite-range action NaN.** The Kinova arm's four continuous-rotation joints have an infinite joint range $[-\infty, +\infty]$. The action-conversion code rescales normalized actions using each joint's range midpoint and half-width, which for these joints evaluate to $\mathrm{NaN}$ and $+\infty$; the guard meant to catch degenerate scales ($\texttt{multiplier} > 0$) does not fire because $\infty > 0$ is true, so every action was poisoned from the first step. Substituting a $[-\pi, \pi]$ range for non-finite joints reduced the NaN rate over $100$ random rollouts from $100/100$ to $1/100$ (the remainder being benign contact-induced cases).

**A CPU–GPU MJX instability.** The hallway base trained cleanly on CPU but produced flat-zero reward on GPU. A motion probe showed that the base position diverged to floating-point overflow within roughly $20$ steps—but only on GPU—whenever the base contacted the cube or a wall. The cause was an unphysically small base inertia: the $61.7$ kg base had a rotational inertia of $0.001$, about three thousand times too small for its size. Any off-center contact produced an enormous yaw acceleration (torque$/I$) that diverged on the MJX-GPU solver while the CPU solver happened to hold. Setting a realistic inertia ($\approx 2.5$, from $m(w^2 + d^2)/12$) eliminated the divergence on both backends. This is a useful reminder that contact-rich whole-body models can be stable on one compute backend and unstable on another.

### 5.8 Constrained-Exploration Composition: Promising but Hard to Learn

Finally, we train the composition policy—the gate plus the ball-constrained residual—on the hallway with the `multi` barrier. Once the simulation findings above were fixed, the pipeline runs end-to-end on GPU and earns nonzero sparse reward from the first iteration. Two ingredients were necessary to get any learning signal at all. Without *gate stickiness*, a zero-initialized gate re-selects a primitive every step and the rollout jitters incoherently, never completing the navigate→push→navigate sequence and earning no reward. Without *reward-to-go* credit and an un-diluted gate gradient, the gate's policy gradient is overwhelmed by the residual term, so the gate fails to move off a near-uniform selection distribution.

With these in place, the composite earns reward through stochastic exploration, but the learned *deterministic* gate does not yet reliably acquire the correct state-conditional sequence: the gate's selection entropy decreases only slowly and the policy has not converged to a high-success composi-

tion by the time of writing. We attribute this to the difficulty of high-level credit assignment from sparse reward—deciding *when* to switch primitives—rather than to low-level execution, which the scripted demonstration shows is already solved by the same frozen primitives. We therefore report the learned composition as a promising but preliminary result, with the scripted demonstration as the working proof that the primitives compose, and leave a converged learned gate (and the full `none/single/euclidean/multi` comparison) to ongoing work.

## 6 Discussion

Our results show that contrastive reinforcement learning is an effective foundation for demo-free sparse-reward primitive learning. On the arm-push task, CRL outperforms SAC and vanilla policy gradient, suggesting that contrastive goal relabeling provides a useful learning signal when dense rewards or demonstrations are unavailable. This supports the use of CRL as a base primitive-learning method for robotic manipulation.

MAB provides a modest improvement when correcting a frozen CRL primitive near its training distribution. The corrected policy improves success on `arm_push_hard`, and qualitative rollouts suggest that the residual preserves the broad structure of the learned primitive while making local adjustments. However, the gains are not uniform: MAB does not consistently improve efficiency or action smoothness, and training is sensitive to checkpoint selection. This suggests that the Mahalanobis constraint can limit residual deviations, but does not by itself guarantee stable or efficient execution.

The main limitation is generalization. On harder manipulation tasks, such as pick-and-place and out-of-distribution start or goal configurations, MAB does not reliably recover performance. This indicates that the bottleneck is not only how to constrain residual actions, but also whether the frozen CRL critic, actor, and learned embedding geometry remain meaningful outside the original training distribution. In particular, embedding-space MAB depends on the CRL representation capturing behaviorally useful distances; when that representation fails to generalize, staying close in embedding space may not preserve useful behavior.

Moving from refining one primitive to composing several reframes where the difficulty lives. In the refinement regime, the low-level correction is easy to optimize and the bottleneck is generalization of the frozen geometry. In the composition regime, low-level execution is *not* the bottleneck—the scripted demonstration shows that the same frozen primitives already solve the hallway—and the difficulty moves up a level, to learning *when* to switch primitives from sparse reward. The Mahalanobis ball plays a clean, shared role in both: it confines a correction to the region a primitive supports, and it confines exploration to motions the primitives already know. But constraining the action space does not by itself solve temporal credit assignment, which is why our learned gate remains preliminary even though the constrained low-level behavior is reliable. Finally, the two simulation findings are a practical lesson in their own right: in contact-rich whole-body simulation, silent failures—a NaN from an infinite joint range, an overflow from an unphysical inertia that appears only on GPU—can masquerade as "the algorithm does not learn," and isolating them required targeted diagnostics rather than reward-curve watching.

## 7 Conclusion

Overall, this project provides an empirical study of demo-free primitive learning with CRL and of a single Mahalanobis-ball mechanism used both to refine one frozen primitive and to compose several. CRL learns strong demo-free pushing primitives from sparse reward on both a fixed arm and a mobile manipulator; MAB improves a frozen primitive locally without retraining it; and the same construction lets a mobile manipulator combine frozen primitives to clear a blocked hallway, which a scripted sequencer over the primitives solves in five of six seeds. The open challenge is learning the high-level composition itself: constraining exploration to the primitives' action geometry is not enough to overcome sparse-reward temporal credit assignment, and our learned gate remains preliminary. Future work includes a converged learned gate and the full barrier ablation, stronger high-level credit assignment, broader training distributions, and replacing the scripted navigation and sequencing with a learned or VLM-driven planner.

## 8 Team Contributions

All team members worked collaboratively on the project design, implementation, experiments, result analysis, figure generation, and final report writing.

- **Dylan Zhou:** Contributed especially to the CRL primitive-learning pipeline, the TidyBot mobile-manipulation environment suite and demo-free push-aside primitive, the constrained-exploration composition trainer and scripted hallway demonstration, and the simulation debugging (the infinite-range NaN and the base-inertia GPU findings).
- **Anuva Banwasi:** Contributed especially to the CRL baseline comparison against vanilla policy gradient and SAC, as well as embedding-space MAB training and rollout evaluation.
- **Jiaye Zou:** Contributed especially to the MAB comparison experiments, including efficiency and smoothness analyses comparing policies with and without MAB.

**Changes from Proposal** The final project expanded the proposal into a concrete empirical study of CRL and the Mahalanobis Action Barrier. We implemented demo-free CRL primitive learning in MuJoCo Franka pushing environments, compared it against vanilla policy gradient and SAC, and developed MAB as a residual correction on top of a frozen CRL primitive, analyzing rollouts, checkpoint behavior, efficiency, and action smoothness. The main change is in how the mobile-manipulation thread is reported: our most quantitative learning results are on fixed-base Franka pushing (CRL validation and MAB refinement), while the TidyBot thread—an MJX mobile-manipulation env suite, a demo-free CRL push-aside primitive, a base-relative deployment, and a constrained-exploration composition trainer—is reported through a working scripted hallway demonstration and a preliminary learned composition rather than a fully converged whole-body policy.

**AI Tools Disclosure** We used an AI coding assistant (Claude) to help with parts of the engineering and debugging, under our direction and review. AI assistance contributed to: diagnostic and visualization scripts; identifying and fixing the infinite-range action-conversion NaN; headless simulation sanity checks; the base-relative deployment of the push primitive; the constrained-exploration composition trainer (gate stickiness, the reward-to-go advantage, and the gate-gradient normalization); diagnosing the CPU–GPU base-inertia instability; and the hallway-demonstration figure script. The research questions, experimental design, method, analysis, and writing are our own, and all AI-assisted code and findings were reviewed and validated by the authors.

## References


Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. 2017. Hindsight Experience Replay. In *Advances in Neural Information Processing Systems*.

Pierre-Luc Bacon, Jean Harb, and Doina Precup. 2017. The Option-Critic Architecture. In *AAAI Conference on Artificial Intelligence*.

Roy Betser, Eyal Gofer, Meir Yossef Levi, and Guy Gilboa. 2026. InfoNCE Induces Gaussian Distribution. In *International Conference on Learning Representations*. Oral.

Benjamin Eysenbach, Tianjun Zhang, Ruslan Salakhutdinov, and Sergey Levine. 2022. Contrastive Learning as Goal-Conditioned Reinforcement Learning. In *Advances in Neural Information Processing Systems*.

Scott Fujimoto, David Meger, and Doina Precup. 2019. Off-Policy Deep Reinforcement Learning without Exploration. In *International Conference on Machine Learning*.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. 2018. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. In *International Conference on Machine Learning*.

Tobias Johannink, Shikhar Bahl, Ashvin Nair, Jianlan Luo, Avinash Kumar, Matthias Loskyll, Juan Aparicio Ojea, Eugen Solowjow, and Sergey Levine. 2019. Residual Reinforcement Learning for Robot Control. In *IEEE International Conference on Robotics and Automation*. IEEE, 6023–6029.

Aviral Kumar, Justin Fu, George Tucker, and Sergey Levine. 2019. Stabilizing Off-Policy Q-Learning via Bootstrapping Error Reduction. In *Advances in Neural Information Processing Systems*.

Tom Schaul, Daniel Horgan, Karol Gregor, and David Silver. 2015. Universal Value Function Approximators. In *International Conference on Machine Learning*.

John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. 2015. Trust Region Policy Optimization. In *International Conference on Machine Learning*.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. In *arXiv preprint arXiv:1707.06347*.

Tom Silver, Kelsey Allen, Josh Tenenbaum, and Leslie Pack Kaelbling. 2018. Residual Policy Learning. arXiv:1812.06298 [cs.LG]

Richard S. Sutton, Doina Precup, and Satinder Singh. 1999. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence* 112, 1–2 (1999), 181–211.

Kevin Wang, Ishaan Javali, Michał Bortkiewicz, Tomasz Trzciński, and Benjamin Eysenbach. 2025. 1000-Layer Networks for Self-Supervised RL: Scaling Depth Can Enable New Goal-Reaching Capabilities. arXiv:2503.14858 [cs.LG]

Ronald J. Williams. 1992. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Machine Learning* 8 (1992), 229–256.

Zhikai Zhang, Haofei Lu, Yunrui Lian, Ziqing Chen, Yun Liu, Chenghuai Lin, Han Xue, Zicheng Zeng, Zekun Qi, Shaolin Zheng, Qing Luan, Jingbo Wang, Junliang Xing, He Wang, and Li Yi. 2026. Learning Athletic Humanoid Tennis Skills from Imperfect Human Motion Data. arXiv:2603.12686 [cs.RO] `https://arxiv.org/abs/2603.12686`